

Universitatea de Stat din Moldova

Facultatea de Fizică și Inginerie

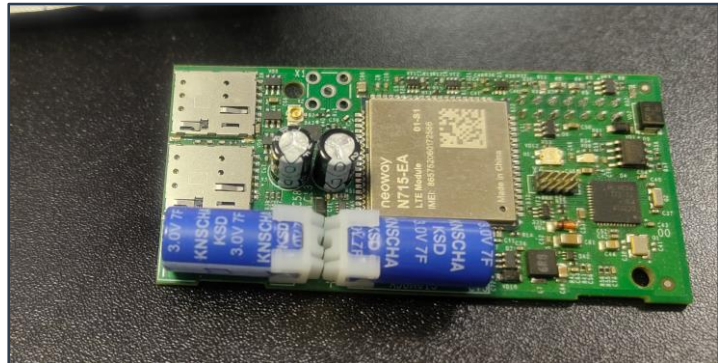
Practica tehnologică

Penzari Marc
TI231

2026

Placa de bază

- MCU: TI CC2652;
- 3GPP modem: Neoway N715;
- 2x sloturi SIM;
- 2x7F condensatori;



Despre dispozitiv

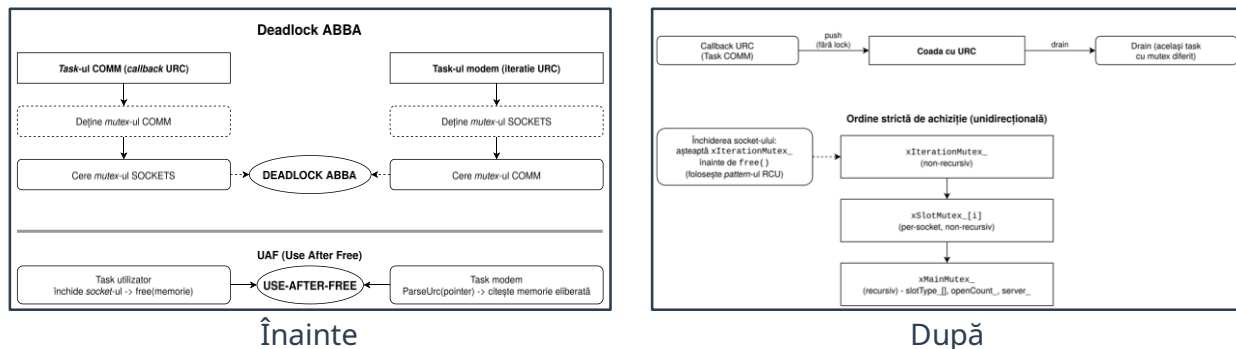
- Interfață de comunicare pentru contor;
- Proiectat pentru mod *plug-in*;
- 3GPP modem <-> HDLC;
- Posibilitatea de rulare temporară fără o sursă de alimentare.



Activitățile realizate

- Proiectarea și realizarea schemei de *mutex*-uri fără interblocaje;
- Reproiectarea *layer*-ului de *socket*-uri (OOP -> DOP);
- Implementarea gestionării packetelor lungi recepționate în mod *inline*;
- Dezvoltarea componentului de rutare a mesajelor recepționate.

Schema de *mutex*-uri fără interblocaje



Înainte:

- 2 task-uri (thread-uri): 1) COMM: de procesare a datelor; 2) MODEM: de gestionarea stării;
- COMM ține mutex-ul COMM; MODEM ține mutex-ul SOCKETS;
- COMM ține COMM, și ia SOCKETS când procesează URC-urile pentru socket-uri;
- MODEM ține SOCKETS, și ia COMM când are nevoie să schimbe starea socket-urilor;
- UAF: Utilizatorul închide socket-ul, și eliberă memoria; Task-ul MODEM/COMM utilizează memoria eliberată în timpul procesării.

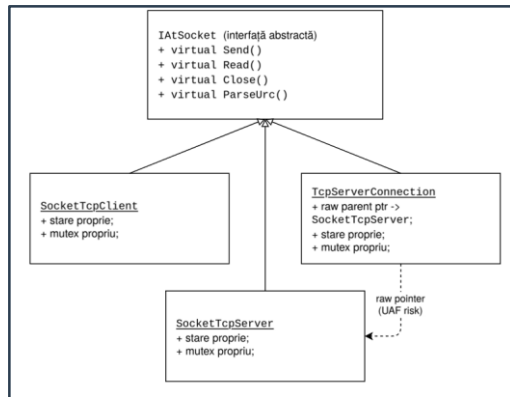
După:

- URC-urile sunt puse de COMM într-o coadă fără utilizarea mutex-urilor (un scriitor -> un cititor),

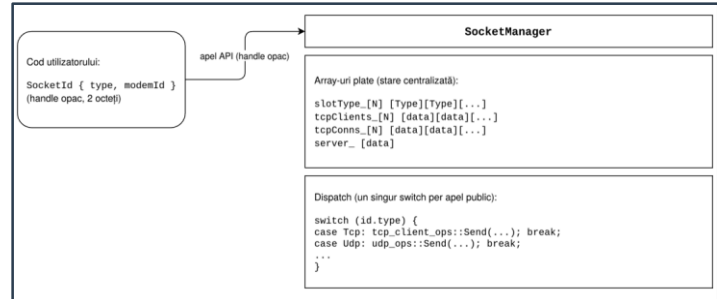
apoi procesate de MODEM;

- Se introduce: mutex de iterare a socket-urilor + fiecare socket își are un mutex pre-definit;
- Ordinea de luare a mutex-urilor este fără interdependențe;
- Riscul UAF nu este eliminat complet. Slide-ul următor.

Reproiectarea *layer*-ului de socket-uri



Înainte



După

6

Înainte:

- Socket-urile sunt obiecte independente, dar gestionate de un manager;
- Complex:
 - Independența obiectelor necesită urmări împletite de stări;
 - Răspunderea la evenimente externe necesită rutare complexă;

După:

- Socket-urile sunt o combinație între AoS și SoA. Starea socket-ului trăiește doar într-un loc: SocketManger;
- User-ul ține doar un ID care identifică o stare în array;
- Beneficii:

- Logică centralizată: starea tuturor socket-urilor este gestionată de un singur obiect într-un singur loc;
- Eficiență: memoria pentru socket-uri este pre-alocată; nu este împrăștiată -> eficiență MCU (cache & pre-fetch);
- Zero risc de UAF: fără alocări și dealocări independente. Doar setarea stării.

Recepționarea packetelor lungi

Problema:

- Recepționarea mesajelor folosind metoda *polling*;
- Pierderea datelor în cazurile:
 - Recepționarea mesajelor lungi (8KB+);
 - Închiderea *socket*-ului în timpul recepției datelor.

Soluția:

- Datele sunt recepționate împreună cu notificarea de recepție;
- S-a introdus un *bypass* al cozii pentru URC-urile de recepție `+TCPRECV/+TCPRECV(S)`;
- Se apelează direct *handler*-ul de recepție a *socket*-ului.

7

- Recepționare polling: dispozitivul primește notificare de la modem despre recepție -> își setează flaguri de stări -> citește datele manual;
- Pierderea mesajelor lungi: buffer-ul modemului este limitat -> citire lentă -> date re-înscrise. Nu este o problemă în TCP, dar este una în UDP;
- Modemul nu permite citirea buffer-ului de recepție după închiderea *socket*-ului. *Socket* închis automat înainte de citire manuală -> date pierdute;
- Recepționarea mesajelor în mod inline necesită o soluție specială: slide-ul următor (PacketPool)

Componentul de rutare a mesajelor

- Ținta primară a dispozitivului: furnizarea comunicării între contorul electric și lumea externă;
- A fost realizat:
 - PacketPool: manager de spațiu dinamic pentru date;
 - BridgeOrchestrator: manager de direcționare a datelor în dependență de adresă.

- PacketPool: o colecție de buffer-uri pre-definite, care pot fi „închiriate” de diferite componente:
 - Socket-uri: recepționează datele inline -> le stochează în buffer-ul închiriat -> pune id-ul buffer-ului în coadă;
 - SerialLink: recepționează datele UART -> fix ca și socket-urile: le stochează în buffer-ul închiriat -> pune id-ul buffer-ului în coadă;
- BridgeOrchestrator:
 - Rutează id-urile din coadă ca pachetele recepționate.

Concluzii

- Concurența și sincronizarea pot fi extrem de complexe;
- OOP nu este unica paradigmă;
- DOP permite soluții eficiente unde OOP restricționează;